Tomato Leaf Disease Classification from Colour and Texture Features

Davide Pomaro — 2191101

August 14, 2026

Note on length. Section 2.1 describes the construction of the tabular dataset from the raw images. As agreed with the instructor by e-mail (29/07/2026), this additional task is not counted towards the 12-page limit. Excluding it, the report is within the limit.

1 Abstract

This project addresses the multi-class classification of tomato leaf diseases from RGB images, encompassing 10 distinct disease categories. The dataset utilized is the Kaggle "Tomato Leaf Disease" dataset, which consists of 18 835 images. There is a strong class imbalance, with a ratio of approximately 5.4:1 between the most and least represented classes. The original training and testing sets were combined and then re-split into an 80/20 ratio to facilitate model evaluation. The approach taken involves handcrafted feature extraction, resulting in a 30-dimensional feature vector composed of 24 HSV color histogram bins and 6 Haralick texture descriptors derived from the Gray Level Co-occurrence Matrix (GLCM). To choose the most effective classification model, four different classifiers were evaluated using 5-fold cross-validation on a stratified subsample of the training set: k-Nearest Neighbors, Support Vector Machine, Decision Tree, and Logistic Regression. Then all the models were trained on the entire training set and evaluated on the test set. k-Nearest Neighbors (k-NN, k=5) was selected as the best performing model; its final evaluation on the test set yielded an accuracy of 0.931 and a macro-F1 score of 0.918. This demonstrates the effectiveness of the handcrafted feature approach in classifying tomato leaf diseases.

2 Dataset

2.1 Building the dataset: from images to a tabular file

This subsection documents the additional task of producing the tabular dataset from raw images, agreed with the course teacher as an extra component of the project. Everything from Section 2.2 onwards follows the standard structure and constraints, starting from the resulting CSV file.

The source of this project is not a table but a collection of photographs, so the tabular dataset used by the machine learning part had to be produced first. The raw material is the "Tomato Leaf Disease" collection available on Kaggle [1]: 18 835 JPEG RGB images of tomato leaves, organised in one folder per class. Three representative images are shown in Figure 1.

To reproduce the extraction, the raw collection must be placed as `tomato_leaf_disease_dataset/Tomato Leaf` relative to the project root; the program also searches up to eight parent directories above the executable, so building in `bin/Debug` or `bin/Release` does not matter. See the project `README` for the exact steps and for running the program without re-downloading the dataset.

Turning an image into a row of numbers means choosing a set of measurements that summarise it.



(a) Early blight **(b)** Leaf mold **(c)** Healthy

Figure 1: Example leaf images for three classes of the dataset.

Two families were combined, because neither is sufficient alone: several diseases produce lesions of similar hue but different structure, while others alter the colour of the whole leaf without changing its surface. The result is the $d = 30$ vector of Table 1.

Table 1: Composition of the feature vector extracted from every image, $d = 30$.

Extractor	Description	Dim.
`HsvHistogramExtractor`	Marginal HSV histograms, 8 bins per channel (3×8)	24
`HaralickTextureExtractor`	GLCM descriptors: contrast, correlation, energy, homogeneity, entropy, dissimilarity	6
`CompositeFeatureExtractor`	Concatenation of the two above	**30**

Before any measurement is taken, every image is decoded and resized to a uniform 128×128. The resize is required because the co-occurrence matrix described below counts pairs of pixels at a fixed distance, which corresponds to a different physical scale on images of different size; it also divides the pixels to be visited by four, and the whole extraction already takes about ten minutes. Unsupported or unreadable files are reported and skipped instead of aborting the run (Section 7.1); nothing was in fact discarded here.

HSV colour features

The three RGB channels of every pixel are scaled to $[0, 1]$ and converted to the HSV representation: hue (H) is the angle in $[0, 360)$ derived from whichever channel is the maximum, set to 0 where it is undefined, that is on achromatic pixels; saturation (S) is the difference between the maximum and the minimum RGB value divided by the maximum, so that it falls in $[0, 1]$; value (V) is simply the maximum of the three channels. HSV was preferred to RGB because it separates the intensity from the chromaticity, which RGB spreads across all three channels [4]: lesion colour is the discriminative signal, and keeping it apart from brightness makes the descriptor less sensitive to how the leaf was lit. The same work reports better retrieval precision and recall for HSV histograms than for RGB ones.

Each value is then quantised into $B = 8$ equal-width bins, hue being divided by 360 beforehand so that each bin spans 45 degrees. One counter per channel is incremented for every pixel; each histogram is then divided by the number of pixels, so that it sums to one and is invariant to image size. The three are concatenated into $3B = 24$ components, `hsv_H_bin0` to `hsv_V_bin7`.

These are three marginal histograms rather than a single joint one. A joint histogram would describe which colours occur together, which the marginal version cannot: it records how much green and how much darkness are present, but not whether the dark pixels were the green ones. It would also be considerably larger, and each of its cells would be estimated from far fewer pixels. The marginal form was preferred because it keeps the feature vector short, which matters for the distance-based classifiers used later.

GLCM/Haralick texture features

The texture features are computed from the Gray Level Co-occurrence Matrix (GLCM). Each image is converted to grayscale with the standard ITU-R BT.601 luminance weights, $0.299R + 0.587G + 0.114B$, and quantised to $L = 16$ gray levels. An $L \times L$ matrix is then accumulated by counting how often two pixels at a fixed offset take a given pair of levels: the offset has distance $d = 1$ and the four orientations 0°, 45°, 90° and 135°, all summed into the same matrix, and every pair is counted in both directions so that the matrix is symmetric. Dividing by the total number of counts turns it into a joint probability distribution $p(i, j)$, from which six of the texture descriptors introduced by Haralick et al. [3] are derived: *contrast*, $\sum_{i,j}(i-j)^2 p(i,j)$, which grows when neighbouring pixels differ strongly, that is for coarse or heavily marked texture; *dissimilarity*, $\sum_{i,j} |i-j|\, p(i,j)$, the same quantity with a linear instead of quadratic penalty; *correlation*, $\left(\sum_{i,j} ij\, p(i,j) - \mu_x\mu_y\right)/(\sigma_x\sigma_y)$, the linear dependency between the levels of the two pixels; *energy*, $\sum_{i,j} p(i,j)^2$, high when a few pairs dominate, that is for uniform texture; *homogeneity*, $\sum_{i,j} p(i,j)/(1+(i-j)^2)$, high when the mass lies close to the diagonal, that is for smooth texture; and *entropy*, $-\sum_{i,j} p(i,j) \log p(i,j)$, the disorder of the distribution.

Two quantisation choices deserve a comment. Reducing the image to 16 gray levels instead of 256 keeps the co-occurrence matrix small: at full depth the same pixel pairs would be spread over a much larger matrix, most of whose cells would be empty, and the statistics derived from it would be unreliable. Summing the four orientations into a single matrix makes the descriptor invariant to the orientation of the leaf in the photograph, which is arbitrary, and keeps this block at six features instead of twenty-four; the cost is that any directionality of the texture is lost. The six descriptors are not independent of one another either, so the information they carry is less than their number would suggest.

The two extractors are finally combined behind a single interface and applied to every image; the resulting vectors are written to `features.csv`, one row per image, with the 30 feature columns followed by the class identifier, the class name and the source path. From here on that file *is* the dataset, and the images are never read again.

2.2 The tabular dataset

The dataset used from here on is the table produced above: 18 835 rows, one per leaf image, each described by $d = 30$ numerical features and labelled with one of $K = 10$ classes, corresponding to nine diseases and to healthy leaves. The class distribution is reported in Table 2.

Table 2: Class distribution. The original `train/` and `test/` folders of the source collection were merged into a single set of 18 835 rows, which is then re-split as described in Section 5.1.

Class	Total	Share	Train (80%)	Test (20%)
Bacterial_spot	2127	11.3%	1702	425
Early_blight	1000	5.3%	800	200
Late_blight	1909	10.1%	1527	382
Leaf_Mold	1000	5.3%	800	200
Septoria_leaf_spot	1771	9.4%	1417	354
Spider_mites Two-spotted_spider_mite	1676	8.9%	1341	335
Target_Spot	1404	7.5%	1123	281
Tomato_Yellow_Leaf_Curl_Virus	5357	28.4%	4286	1071
Tomato_mosaic_virus	1000	5.3%	800	200
healthy	1591	8.5%	1273	318
Total	**18835**	**100%**	**15069**	**3766**

The dataset is markedly imbalanced. The modal class, Tomato_Yellow_Leaf_Curl_Virus,

accounts for 28.4% of the rows and holds 5.4 times as many samples as each of the three smallest classes. Those three contain exactly 1000 images each, a round figure across three unrelated diseases that indicates a truncation applied when the collection was assembled rather than a property of the phenomenon. This imbalance motivates three choices made later in the project:

- stratified sampling when splitting into training and test sets, so that each class keeps its proportion in both (Section 5.1);
- the macro-averaged F1 score, rather than accuracy alone, as the criterion for model selection and evaluation (Sections 5.2 and 6);
- an analysis of the confusion matrix, since aggregate figures hide the behaviour of the minority classes (Section 6).

2.3 Descriptive statistics of the features

Table 3 reports mean, standard deviation, minimum and maximum of each of the 30 features over the whole dataset. Three observations follow from it.

First, hue is concentrated in bins 0, 1, 2 and 7 (roughly three quarters of the average histogram), matching leaves that are green, yellow-green or, once diseased, yellow-brown; bin 0 is inflated further because achromatic pixels are also assigned a hue of zero. Second, saturation is skewed towards its first bin (mean 0.451), since dark background and shadow make up a large share of every image.

Third, and most relevant for the following sections, HSV and GLCM features live on incompatible scales: the HSV bins are proportions in $[0, 1]$ with standard deviations of at most 0.219, while the GLCM descriptors are unbounded, e.g. contrast reaches 12.142 with a standard deviation of 2.040. Without standardisation a distance-based classifier would be dominated by contrast and entropy, ignoring the 24 colour features. This justifies the z-score normalisation applied in Section 4.2.

Table 3: Descriptive statistics of the 30 features over all 18 835 rows.

Feature	Mean	SD	Min	Max	Feature	Mean	SD	Min	Max
hsv_H_bin0	0.161	0.215	0.000	0.939	hsv_S_bin7	0.022	0.032	0.000	0.729
hsv_H_bin1	0.222	0.219	0.000	0.985	hsv_V_bin0	0.031	0.068	0.000	0.921
hsv_H_bin2	0.205	0.122	0.000	0.698	hsv_V_bin1	0.068	0.053	0.000	0.580
hsv_H_bin3	0.038	0.072	0.000	0.497	hsv_V_bin2	0.131	0.088	0.000	0.603
hsv_H_bin4	0.017	0.060	0.000	0.811	hsv_V_bin3	0.242	0.113	0.005	0.786
hsv_H_bin5	0.092	0.169	0.000	0.847	hsv_V_bin4	0.314	0.115	0.000	0.827
hsv_H_bin6	0.097	0.153	0.000	0.826	hsv_V_bin5	0.168	0.110	0.000	0.753
hsv_H_bin7	0.168	0.206	0.000	0.887	hsv_V_bin6	0.043	0.066	0.000	0.622
hsv_S_bin0	0.451	0.181	0.000	0.972	hsv_V_bin7	0.004	0.018	0.000	0.553
hsv_S_bin1	0.178	0.165	0.001	0.837	glcm_contrast	2.598	2.040	0.123	12.142
hsv_S_bin2	0.121	0.094	0.002	0.839	glcm_correlation	0.789	0.119	0.221	0.981
hsv_S_bin3	0.090	0.058	0.000	0.731	glcm_energy	0.053	0.054	0.010	0.823
hsv_S_bin4	0.066	0.044	0.000	0.453	glcm_homogeneity	0.628	0.125	0.319	0.960
hsv_S_bin5	0.045	0.044	0.000	0.332	glcm_entropy	3.659	0.618	0.666	4.942
hsv_S_bin6	0.028	0.038	0.000	0.268	glcm_dissimilarity	1.001	0.472	0.102	2.637

Figure 2 shows how two representative features are distributed across the ten classes. The colour feature separates Tomato_Yellow_Leaf_Curl_Virus clearly from every other class, as expected from a disease whose main symptom is a diffuse yellowing of the leaf, while the texture feature isolates healthy leaves, whose intact venation produces far more local intensity variation than a lesioned surface. Neither feature separates all ten classes on its own, which is precisely why a multivariate classifier is required, but both carry evident signal.

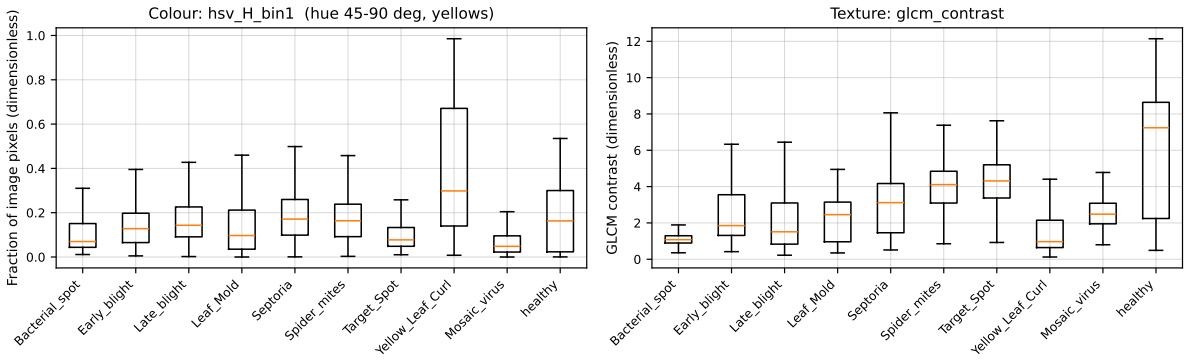


Figure 2: Distribution of one colour feature (left) and one texture feature (right) across the ten classes. Boxes span the interquartile range, the line marks the median, whiskers extend to 1.5× IQR; outliers are omitted for readability.

3 Problem

Following the formulation adopted in the course, classification is the problem of estimating a discrete label function $f : \mathcal{X} \to \mathcal{C}$, where $\mathcal{X}$ is the feature space and $\mathcal{C}$ the set of classes. We are given m i.i.d. samples

$$Z = (\mathbf{x}_1, c_1), \ldots, (\mathbf{x}_m, c_m), \qquad \mathbf{x}_i \in \mathcal{X} \subseteq \mathbb{R}^d, \quad c_i \in \mathcal{C}, \tag{1}$$

drawn from an unknown distribution $\mathcal{D}$, and we look for a hypothesis $h \in \mathcal{H}$ that approximates f. The discrepancy between the two is measured by the 0–1 loss, whose empirical value on Z is

$$\ell_{0\text{-}1}(h(\mathbf{x}), c) = \begin{cases} 1 & \text{if } h(\mathbf{x}) \neq c, \\ 0 & \text{otherwise,} \end{cases} \qquad \hat{L}(h) = \frac{1}{m} \sum_{i=1}^{m} \ell_{0\text{-}1}(h(\mathbf{x}_i), c_i). \tag{2}$$

What we actually care about is the expected loss $L_{\mathcal{D},f}(h) = \mathbb{E}_{(\mathbf{x},c)\sim\mathcal{D}}[\ell_{0\text{-}1}(h(\mathbf{x}), c)]$, which is unknown and is estimated on a test set held out from Z (Section 5.1).

In this project $d = 30$: 24 marginal HSV histogram bins (3 channels × 8 bins) and 6 Haralick texture descriptors, as summarised in Table 1. The dataset contains $m = 18\,835$ samples and $\mathcal{C} = \{0, 1, \ldots, 9\}$, so $|\mathcal{C}| = 10$; the identifiers are those assigned by `LabelEncoder` in alphabetical order of the class name, and they index the rows and columns of the confusion matrix of Table 8. Since every leaf carries exactly one condition, the classes are mutually exclusive and the task is supervised, multi-class and single-label.

Minimising $\hat{L}$ alone is not a sufficient criterion here. The classes are strongly unbalanced, from 28.4% of the samples for the largest to 5.3% for each of the three smallest, so a hypothesis that systematically ignored the rare classes would still attain a low empirical loss. Model selection and evaluation are therefore driven by the macro-averaged F1 score, which weights every class equally regardless of its support, and which is defined formally in Section 6.

4 Data Preprocessing

4.1 Data cleaning

The dataset enters this stage as the tabular file produced in Section 2.1, and it requires no cleaning: the 18 835 rows are complete, with no missing value in any of the 30 feature columns and no duplicate row, because every row is generated programmatically from one distinct image file. This was verified rather than assumed: the number of image files across the class folders of

the source collection is 18 835, exactly the number of rows the extraction produced, so no image was silently discarded. The only reading that can fail is a malformed number, which the CSV parser reports as a `DatasetException` naming the offending line; this never occurred on the file used here.

Two properties of the labels are worth stating, since the rest of the pipeline depends on them. The class of each row is derived from the name of the folder the image came from, and `LabelEncoder` maps those names to integer identifiers *in alphabetical order*: the mapping is therefore deterministic, and identifier c denotes the same class in every execution and in every table of this report. All numeric conversions, on writing and on reading, are performed with `CultureInfo.InvariantCulture`, so that the file is interpreted identically regardless of the decimal separator configured on the machine.

4.2 ML specific preprocessing

All the features are standardized using z-score normalization, which transforms each feature to have a mean of zero and a standard deviation of one. The normalization parameters (mean and standard deviation) are computed solely on the training set to prevent data leakage, and then applied to both the training and test sets. This ensures that the model does not gain any information from the test set during training. During cross-validation, the normalizer is refitted for each fold to maintain this separation. Standardization is particularly important for distance-based classifiers like KNN and SVM, as features with larger scales can dominate the distance metric, leading to biased results. The z-score normalization is defined mathematically as follows:

$$z = \frac{x - \mu}{\sigma} \tag{3}$$

where x is the original feature value, μ is the mean of the feature in the training set, and σ is the standard deviation of the feature in the training set.

5 Data Mining

5.1 Train and test split

Before any training, the dataset is divided into a training set and a test set. The split is *stratified*: each class is divided separately, so that both sets contain the same proportion of every class as the full dataset. The training set holds 15 069 images (80%) and the test set 3 766 (20%); the exact counts per class are given in Table 2.

Stratification was chosen because the classes are very unequal in size. The three smallest contain 1 000 images each, against 5 357 for Tomato_Yellow_Leaf_Curl_Virus. A purely random split gives no guarantee on the small classes: it could leave one of them with clearly fewer than the expected 200 test images, and their precision and recall would then be estimated from very few cases and would vary noticeably from one run to the next. Stratification removes this source of variability, and is reported to reduce both the bias and the variance of the resulting estimates with respect to a non-stratified partition [5].

The splitter uses a fixed random seed (42), so the same partition is produced at every execution and all the results reported here can be reproduced exactly. Once the split is done the test set is set aside and is not used again until the final evaluation in Section 6.

5.2 Model selection / Hyperparameter tuning

Four models are compared, and the comparison deliberately does not use the test set. The test set has a single purpose: to give an unbiased estimate of how the chosen model will behave on

data it has never seen. If the four models were compared on it and the best one kept, part of that model's advantage would simply be luck on those particular 3 766 images, and the score reported afterwards would be optimistically biased. This is a form of overfitting, moved from the level of the parameters to the level of the choice of the model. The selection is therefore made on the training data alone.

The procedure is a 5-fold cross-validation. The training set is divided into five parts of equal size, again keeping the class proportions; each model is then trained five times, on four parts, and evaluated on the part left out. Four models times five folds gives twenty trainings in total. The normaliser is refitted inside every fold, using only the four training parts, so that no information from the validation part can leak into it (Section 4.2). The criterion is the mean macro-F1 over the five folds, and the results are reported in Table 5.

Cross-validation was preferred to a single validation split for two reasons [5]. First, every training image is used for validation exactly once, so no data is set aside permanently. Second, repeating the measurement five times gives a standard deviation as well as a mean, and that is what makes the comparison meaningful: the standard deviations lie between 0.006 and 0.016, while the smallest gap between two models, the 0.045 separating k-NN from multinomial logistic regression, is at least three times larger than either of them. The ranking is therefore a real difference between the models and not an accident of how the folds happened to fall. With a single validation split there would be one number per model and no way to make this argument.

Two limitations should be stated. The cross-validation runs on a stratified subsample of the training set, capped at 4 000 instances (3 999 in practice), because running twenty trainings on the full 15 069 would be prohibitively slow. Model selection therefore uses less data than the final fit. Second, no hyperparameter tuning was carried out: the number of neighbours is fixed at $k = 5$, the SVM uses the default settings of the SMO solver and the decision tree is not pruned. What is compared here are four *models*, not four configurations, and this is a limitation revisited in Section 8.

5.3 Machine learning model

The four candidates were chosen so as to cover clearly different families: one rule-based method, one distance-based method and two linear methods.

- **Decision Tree (C4.5).** The feature space is partitioned recursively; at each step the algorithm picks the feature and the threshold that maximise the gain ratio, producing axis-parallel boundaries. It makes no assumption about the scale of the features and can represent very irregular regions, but for the same reason it overfits easily when it is allowed to grow without pruning.
- **k-Nearest Neighbours ($k = 5$).** A lazy learner: training consists only of storing the training instances, and all the work is done at prediction time, when the five closest instances in Euclidean distance vote on the label. It assumes that being close in the feature space means belonging to the same class, and it makes no assumption at all about the shape of the boundary between classes. Because it relies on distances, it requires features on comparable scales, which is what the standardisation of Section 4.2 provides.
- **Linear Support Vector Machine.** It looks for the hyperplane that separates two classes with the widest possible margin, and the ten classes are handled with a one-versus-one scheme. The soft-margin formulation tolerates misclassified points, so perfect linear separability is not required, but the boundary it can express is still a hyperplane. Training uses Sequential Minimal Optimisation (SMO).
- **Multinomial Logistic Regression.** It models the probability of each class with a softmax

over linear combinations of the features, and is fitted by maximum likelihood with the BFGS optimiser. Unlike the other three it returns a probability for every class, not only the predicted label. It assumes that the log-odds are linear in the features.

k-NN was the model selected by the cross-validation, and this result has a structural explanation rather than being a lucky outcome. Distance-based methods usually suffer from the curse of dimensionality: as the number of features grows, distances between points become increasingly similar and the notion of "nearest neighbour" loses its meaning. Here the feature vector has only $d = 30$ dimensions, which is low enough for that effect not to appear, and the training set contains 15 069 instances, so the neighbourhood of a new point is densely populated. At the same time k-NN is the only candidate that imposes no shape on the decision boundary, which matters because the two linear models turn out to be unable to separate these classes well (Section 6). Its cost, however, is paid entirely at prediction time, and this is discussed in Section 7.2.

5.4 Accord library

The Accord.NET framework [2] was used for the four classifiers, and it is the only external library the project depends on. Table 4 lists the namespaces and the classes actually employed.

Table 4: Accord.NET components used in the project.

Namespace	Classes used	Our class
`Accord.MachineLearning`	`KNearestNeighbors`	`KnnClassifier`
`Accord.MachineLearning.DecisionTrees(.Learning)`	`DecisionTree`, `C45Learning`	`DecisionTreeClassifier`
`Accord.MachineLearning.VectorMachines(.Learning)`	`MulticlassSupportVectorMachine`, `MulticlassSupportVectorLearning`, `SequentialMinimalOptimization`	`SvmClassifier`
`Accord.Statistics.Kernels`	`Linear`	`SvmClassifier`
`Accord.Statistics.Models.Regression(.Fitting)`	`MultinomialLogisticRegression`, `MultinomialLogisticLearning`	`MultinomialLogisticClassifier`
`Accord.Math.Optimization`	`BroydenFletcherGoldfarbShanno`	`MultinomialLogisticClassifier`

In every case the pattern is the same: the learning algorithm is configured, its `Learn` method is called on the training matrix and the label vector, and the resulting model exposes a `Decide` method that returns the predicted class for one instance or for a whole matrix.

Everything outside the classifiers themselves was implemented from scratch: the two feature extractors `HsvHistogramExtractor` and `HaralickTextureExtractor`, including the computation of the co-occurrence matrix, together with `ZScoreNormalizer`, `StratifiedSplitter`, `CrossValidator` and `ConfusionMatrix`. Neither extractor references Accord at all.

Accord types never appear outside the `Models` namespace, because every classifier is exposed through the `IClassifier` interface described in Section 7.1. The rest of the program works with that interface only. This keeps the dependency confined to four small classes: replacing the library, or adding a classifier written by hand, requires no change anywhere else.

6 Results

The models are evaluated with five metrics. **Accuracy** is the proportion of correctly classified instances over the total. **Precision** is the proportion of true positives among the instances predicted as positive, **recall** the proportion of true positives among the instances that are

actually positive, and the **F1 score** their harmonic mean; the **macro-F1** averages the per-class F1 treating all classes equally, regardless of their support:

$$\mathrm{P} = \frac{TP}{TP + FP}, \quad \mathrm{R} = \frac{TP}{TP + FN}, \quad F1 = \frac{2\,\mathrm{P} \cdot \mathrm{R}}{\mathrm{P} + \mathrm{R}}, \quad \text{Macro-F1} = \frac{1}{K} \sum_{c=1}^{K} F1_c, \tag{4}$$

with $K = 10$ the number of classes. Since the task is multi-class, precision, recall and F1 are computed *per class* in a one-versus-rest fashion: for a given class c, TP counts the instances of c that were predicted as c, FP the instances of other classes predicted as c, and FN the instances of c predicted as something else. The values reported in Table 7 follow this convention, and the macro average is their unweighted mean over the ten classes. As anticipated in Section 3, the macro-F1 is the criterion of interest here, precisely because it does not let the majority class dominate the judgement.

Table 5: Model selection: 5-fold cross-validation on a stratified subsample of 3999 training instances. Values are the mean over the folds, ± one standard deviation. **This table alone determined the choice of the final model**; the test set played no part in it.

Model	**CV accuracy**	**CV macro-F1**
Decision Tree (C4.5)	0.736 ± 0.015	0.697 ± 0.016
***k*-NN** ($k = 5$)	$\mathbf{0.884 \pm 0.010}$	$\mathbf{0.865 \pm 0.013}$
SVM (linear)	0.828 ± 0.008	0.794 ± 0.013
Multinomial Logistic Regression	0.851 ± 0.005	0.820 ± 0.006

Table 6: Final training on the full training set (15 069 instances) and evaluation on the held-out test set (3766 instances). Reported for completeness: these figures did not influence the selection, which was made on Table 5. The selected model is in bold.

Model	**Train acc.**	**Test acc.**	**Test macro-F1**	**Train time (s)**
Decision Tree (C4.5)	0.999	0.828	0.801	539.5
***k*-NN** ($k = 5$)	**0.984**	**0.931**	**0.918**	**0.1**
SVM (linear)	0.860	0.870	0.843	17.8
Multinomial Logistic Regression	0.858	0.872	0.849	21.4

As we can see in Table 5, the k-NN model with $k = 5$ achieved the highest cross-validation accuracy and Macro-F1 score among the evaluated models. This model was then trained on the full training set and evaluated on the held-out test set, as shown in Table 6. The results confirm that the k-NN model outperforms the other models in terms of both accuracy and Macro-F1 score. More than that, the ranking produced by the cross-validation — k-NN, then multinomial logistic regression, then SVM, then the decision tree — is exactly the ranking observed on the test set. This is the empirical confirmation that the selection procedure worked: the test set was needed to quantify the performance of the chosen model, not to choose it. Its training time is far lower than the decision tree's simply because k-NN stores the training instances instead of fitting anything, and the reported figure does not include prediction on the test set, which for this model is where the cost actually falls (Section 7.2). Another thing to highlight is that the cross-validation results on k-NN (0.865) are lower than the final test results (0.918). This is not a contradiction; during cross-validation, the model is trained on approximately 3200 instances (4/5 of 3999), while in the final training, it is trained on the full 15069 instances. The performance of k-NN improves with more available neighbors, which explains the increase in accuracy and Macro-F1 score on the test set.

Analysing also the other models, we can observe that the Decision Tree model exhibits significant overfitting, with a training accuracy of 0.999 compared to a test accuracy of 0.828. In contrast,

the SVM and Multinomial Logistic Regression models show underfitting, with similar training and test accuracies around 0.86-0.87, indicating that they are not capturing the complexity of the data.

Table 7: Per-class performance of the selected model on the held-out test set.

Class	Precision	Recall	F1	Support
Bacterial_spot	0.942	0.920	0.931	425
Early_blight	0.885	0.770	0.824	200
Late_blight	0.898	0.853	0.875	382
Leaf_Mold	0.924	0.855	0.888	200
Septoria_leaf_spot	0.949	0.946	0.948	354
Spider_mites Two-spotted_spider_mite	0.878	0.925	0.901	335
Target_Spot	0.915	0.922	0.918	281
Tomato_Yellow_Leaf_Curl_Virus	0.947	0.982	0.964	1071
Tomato_mosaic_virus	0.920	0.975	0.947	200
healthy	0.984	0.981	0.983	318
Macro average	0.924	0.913	0.918	3766

Overall accuracy: 0.931 (3766 test samples).

Table 8: Confusion matrix of the selected k-NN model on the test set. Rows are true classes, columns predicted ones. Diagonal entries in bold.

True \ Pred.	0	1	2	3	4	5	6	7	8	9
0 Bacterial_spot	**391**	2	4	0	0	3	2	22	1	0
1 Early_blight	9	**154**	17	0	3	7	3	6	0	1
2 Late_blight	5	12	**326**	7	5	7	4	15	0	1
3 Leaf_Mold	1	1	6	**171**	3	7	2	3	3	3
4 Septoria	1	0	5	2	**335**	2	2	1	6	0
5 Spider_mites	0	2	0	1	3	**310**	6	10	3	0
6 Target_Spot	3	0	2	1	1	10	**259**	1	4	0
7 Yellow_Leaf_Curl	5	3	2	2	1	5	1	**1052**	0	0
8 Mosaic_virus	0	0	1	0	2	1	0	1	**195**	0
9 healthy	0	0	0	1	0	1	4	0	0	**312**

Table 7 reports the per-class behaviour of the selected model and Table 8 shows where its errors go. Four readings follow.

The weakest class is Early_blight, whose recall of 0.770 means that 46 of its 200 test instances are assigned elsewhere, 17 of them to Late_blight, with 12 instances of Late_blight travelling the other way. This is the most reasonable confusion in the whole matrix: both are blights producing brown necrotic lesions, and colour and texture alone do not separate them.

The single largest off-diagonal cell, however, lies elsewhere: 22 instances of Bacterial_spot are predicted as Tomato_Yellow_Leaf_Curl_Virus. Both diseases yellow the leaf, and the 24 marginal HSV bins record that the leaf *contains* yellow without recording *where* the yellow is. This is exactly the limitation of marginal over joint histograms discussed in Section 2.1, and it is therefore a limit of the features rather than of the classifier.

Column 7 also collects 59 false positives, more than any other. That class holds 28.4% of the training rows (Table 2), so with $k = 5$ the probability that the neighbours of an uncertain point belong to it is structurally higher. Its precision remains 0.947, so the damage is contained, but this column is the imprint of the class imbalance on the classifier.

At the opposite end, healthy (F1 0.983) and Tomato_Yellow_Leaf_Curl_Virus (F1 0.964) are

the easy classes: one carries no lesion at all, the other a very characteristic diffuse yellowing, and coarse colour features suffice for both.

7 Software

7.1 OOP Design and implementation

The program is organised into nine namespaces, each with a single responsibility: `Data` loads the images, `Features` describes them, `ML` prepares and validates the data, `Models` wraps the classifiers, `Evaluation` measures the results, `Output` writes the CSV, `Pipeline` orchestrates the stages, `Exceptions` defines the error types and `Util` holds the configuration constants. The first two implement the extra dataset-building task of Section 2.1, and are described here as well because their design follows the same scheme as the rest and gives one of its clearest examples. Figure 3 shows the class hierarchies and Figure 4 how the pieces collaborate at run time.

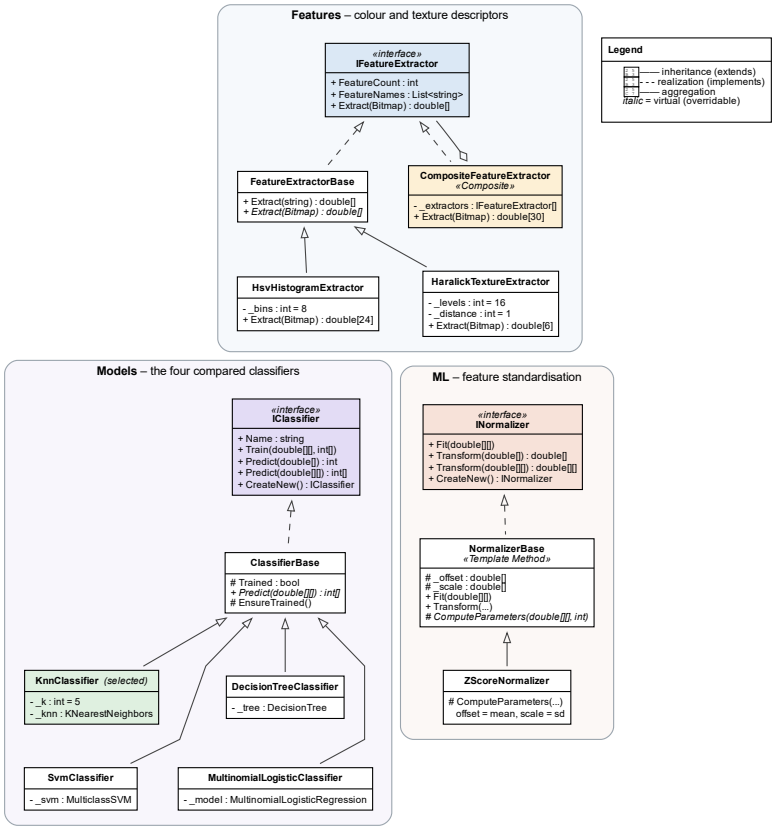


Figure 3: Class hierarchy of the three main abstractions. Each group follows the same scheme: an interface defines the contract, a base class holds the code shared by all implementations and declares as `virtual` the members that vary, and the concrete classes override only what is specific to them.

Interface plus virtual base

The same pattern appears three times, in `Features`, `Models` and `ML`, and is the backbone of the design. The interface states *what* an implementation must offer and is the only type the rest of the program refers to; the base class provides the behaviour that would otherwise be rewritten identically in every subclass. `FeatureExtractorBase`, for instance, implements the overload taking a file path, which opens the image and delegates to the virtual overload

taking a bitmap: the file handling is written once and every extractor only has to describe a bitmap. `ClassifierBase` keeps the `Trained` flag, offers `EnsureTrained()` so that using an untrained model raises a clear exception instead of a null reference, and supplies a default batch `Predict` looping over the single-sample version, which all four Accord-based classifiers override with the more efficient matrix call offered by the library. The members that every model must define, `Train`, `Predict` and `CreateNew`, are declared `virtual` with a body that raises `NotSupportedException`, so a subclass that forgets one fails immediately and with an explicit message; `Name` instead has a genuinely useful default, the class name, which `KnnClassifier` refines to expose its `k`. The constructor of each base class is `protected`: the class can only be reached through a subclass, never instantiated on its own.

Design patterns

Three classical patterns carry most of the flexibility of the project. **Composite**: the composite extractor implements `IFeatureExtractor` and at the same time holds an array of them, concatenating the vectors its parts return, so a group of extractors behaves exactly like a single one and no code downstream is aware that the 30-dimensional vector is made of two blocks. **Template Method**: `NormalizerBase` implements `Fit` and `Transform` and declares one virtual method, `ComputeParameters`; the algorithm lives in the base class and the subclass only decides what the offset and the scale are, which for `ZScoreNormalizer` are the mean and the standard deviation of each column. **Prototype**: `IClassifier` and `INormalizer` declare `CreateNew()`, returning a fresh instance of the same concrete type. This exists for cross-validation, where every fold needs a clean model and a clean normaliser obtained without knowing which class is being held; without it, `CrossValidator` would have to test the runtime type and call the right constructor, and every new model would force a change inside it.

The consequence is that adding a component is local. A third descriptor is one line in `FeaturePipeline`; a fifth classifier is a new class plus one line in `BuildModels()`, after which comparison, cross-validation, selection and reporting all keep working unchanged, because `CrossValidator` and `TrainingPipeline` never mention a concrete type.

Single point of truth

`FeaturePipeline.BuildExtractor()` is the only place that decides how the feature vector is composed, so the CSV and the code reading it cannot disagree on the number or the order of the columns. The same principle governs `Util.Constants`, which holds every configurable value: image size, number of bins, gray levels, train ratio, random seed, number of folds, subsample cap and number of neighbours. No magic numbers are scattered through the code, and reproducing a different configuration means editing one file.

Separation of layers and error handling

Classes in `Data`, `Features`, `ML`, `Models` and `Evaluation` never write to the console: when something goes wrong they throw. Only `Program` and the `Pipeline` classes produce output, which keeps the domain classes usable outside a console application and concentrates error handling in one place. Two custom exceptions carry the cases specific to this project: `DatasetException`, raised when the dataset cannot be located or a class folder is empty, and `InvalidImageFormatException`, raised by `ImageSample` when a file cannot be decoded, carrying the offending path. The extraction loop catches the second one, reports the file and skips it, because a single corrupted JPEG must not waste ten minutes of work; everything else travels up to `Program.Main`, the only place with a `try/catch`, which turns each exception type into a readable message.

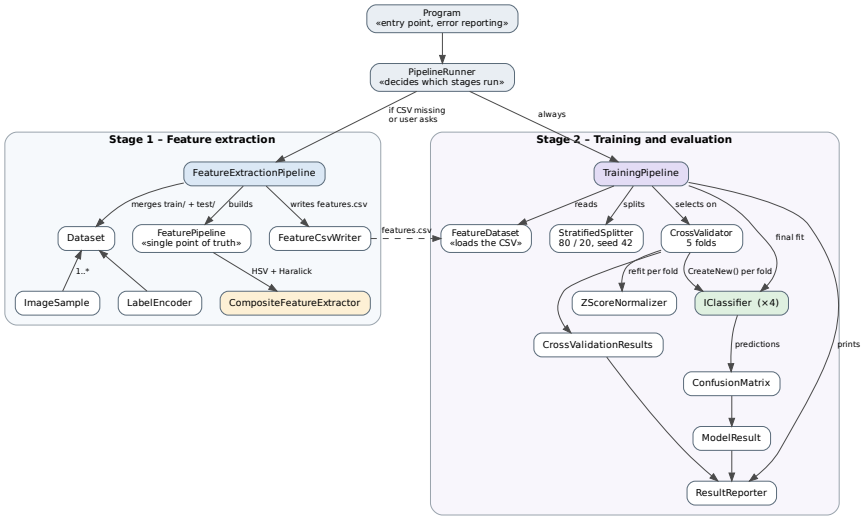


Figure 4: Collaboration between the two pipeline stages and the classes they use. The two stages communicate only through `features.csv`, so the extraction can be run once and the training repeated as often as needed.

The pipeline is deliberately split in two. Extraction reads the images and writes `features.csv`; training reads that file and never touches the images again, so the expensive stage runs once while the cheap one can be repeated at will. Since extraction takes about ten minutes and training a few seconds, at startup `PipelineRunner` detects an existing CSV and asks whether to reuse or rebuild it, reporting its size and date so that the choice can be made knowingly.

7.2 Data structures, algorithms, and computational complexity

Every container was chosen on two criteria: whether its size is known in advance, and what the dominant access pattern is.

- **`double[][]` (jagged) for the feature matrix.** A jagged array is a vector of references to rows, so a row can be copied into a fold without duplicating its content: `StratifiedSplitter` and `CrossValidator` build their subsets in $O(n)$ instead of $O(n\,d)$. It is also the format Accord expects.
- **`int[,]` for the confusion matrix.** The opposite argument: the size is fixed at $K \times K$, rows are never handled individually, and access is always by pair of indices, so a rectangular array is more compact and gives $O(1)$ access.
- **`List<T>` in `Dataset` and for the results.** The number of files is unknown until the directories are scanned, so the container must grow; `List<T>` amortises growth to $O(1)$ per insertion.
- **`Dictionary<string,int>` in `LabelEncoder`.** A class name is translated almost nineteen thousand times, and a hash table gives $O(1)$ per lookup against the $O(K)$ of a linear scan. The reverse direction is a `List<string>` indexed by the identifier, $O(1)$ without hashing.
- **`string[]` for feature and class names.** Fixed in number, read only, and read by position: the simplest structure that satisfies all three conditions.

With n images, $d = 30$ features, $K = 10$ classes, $P = 128^2$ pixels per image, $L = 16$ gray levels and $k = 5$ folds, the main costs are: $O(nP)$ for the HSV histograms and $O(n(P + L^2))$

for the GLCM, dominated by the pixels since $L^2 \ll P$; $O(n\,d)$ for reading the CSV and for standardisation; $O(n)$ for the stratified split, which groups indices by class and shuffles them with the in-place Fisher–Yates shuffle [6] instead of sorting; and the cost of one training multiplied by k for each of the four models in cross-validation, that is twenty trainings, which is precisely why the subsample is capped at 4 000 instances. The two classifiers at the extremes are k-NN, whose training is $O(1)$ beyond storing the references but whose every prediction costs $O(n_{train}\,d)$, and C4.5, which must order the values of each continuous feature at every node, roughly $O(d\,n \log n)$ per level of the tree.

The training times in Table 6 match this analysis. The decision tree is the slowest to train, the two linear models are much faster, and k-NN is almost immediate: it only stores the training data instead of learning anything from it. This column, however, measures the training phase alone. k-NN does its work later, when every new image has to be compared with all the stored ones, so a short training time does not mean a fast model. Most of the total running time of the program is spent extracting the features, not training the models.

8 Conclusion

In the end, the model achieved an overall accuracy of 0.931, which is high, but the recall for the Early Blight class was only 0.770. This means that in a production setting, nearly one in four cases of Early Blight would be missed, which is a significant concern in agricultural applications where false negatives can have more severe consequences than false positives.

With more time and resources, several improvements could be made to enhance the model's performance. These include hyperparameter tuning for the KNN, SVM, and decision tree models, conducting cross-validation on the entire training set without the 4,000-instance cap and comparing the result of this project with the result of a CNN baseline, which was outside the scope of this course.

References

[1] N. Gull, *Tomato Leaf Disease* dataset, Kaggle. `https://www.kaggle.com/datasets/naveedgull/tomato-leaf-disease` (accessed August 2026).

[2] C. R. Souza et al., *The Accord.NET Framework*, 2014. `http://accord-framework.net`

[3] R. M. Haralick, K. Shanmugam, I. Dinstein, *Textural Features for Image Classification*, IEEE Transactions on Systems, Man, and Cybernetics, vol. SMC-3, no. 6, pp. 610–621, 1973.

[4] S. Sural, G. Qian, S. Pramanik, *Segmentation and Histogram Generation using the HSV Color Space for Image Retrieval*, Proceedings of the IEEE International Conference on Image Processing (ICIP), vol. 2, pp. 589–592, 2002. `https://www.cse.msu.edu/~pramanik/research/papers/2002Papers/icip.hsv.pdf`

[5] R. Kohavi, *A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection*, Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI), pp. 1137–1143, 1995. `https://www.ijcai.org/Proceedings/95-2/Papers/016.pdf`

[6] R. Durstenfeld, *Algorithm 235: Random Permutation*, Communications of the ACM, vol. 7, no. 7, p. 420, 1964. `https://doi.org/10.1145/364520.364540`